1. Clone the Remote Repository

The first step is to create a local copy of the shared remote repository. The `git clone` command downloads the project from GitHub to your local machine.

```
$ git clone https://github.com/shmworks/MLOPS_lab8
```

```
mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git clone https://github.com/shmWorks/MLOPs_lab8
Cloning into 'MLOPs_lab8'...
remote: Enumerating objects: 26, done.
remote: Counting objects: 100% (26/26), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 26 (delta 2), reused 26 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (26/26), 7.86 KiB | 1.12 MiB/s, done.
Resolving deltas: 100% (2/2), done.
```

2. Setting Up the Remote URL

To ensure you are pushing to the correct repository, you can set the remote URL using `git remote set-url`.

```
$ git remote set-url origin https://github.com/shmworks/MLOPS_lab8.git
```

You can verify the new remote URL with `git remote -v`.

```
mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git remote -v
origin  https://github.com/UsmanAnwer7086/ml1_9a (fetch)
origin  https://github.com/UsmanAnwer7086/ml1_9a (push)

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git remote set-url origin https://github.com/shmWorks/MLOPs_lab8.git

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git remote -v
origin  https://github.com/shmWorks/MLOPs_lab8.git (fetch)
origin  https://github.com/shmWorks/MLOPs_lab8.git (push)
```

3. Pushing the Initial `main` Branch

After cloning, you may need to push the local `main` branch to the remote repository to establish tracking.

```
$ git push -u origin main
```

```
mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git push -u origin main
Enumerating objects: 26, done.
Counting objects: 100% (26/26), done.
Delta compression using up to 8 threads
Compressing objects: 100% (20/20), done.
Writing objects: 100% (26/26), 7.86 KiB | 1.31 MiB/s, done.
Total 26 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/shmWorks/MLOPs_lab8.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

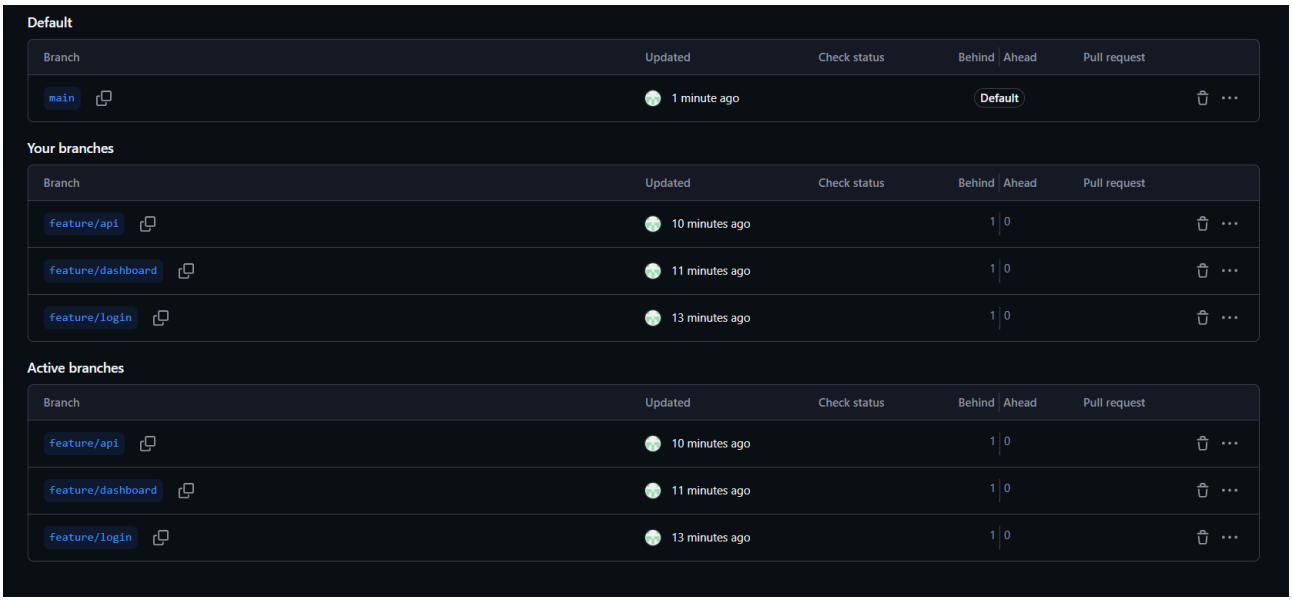
4. Create and Manage Feature Branches

To work on new features without affecting the main codebase, developers create separate branches. Here, three feature branches (`feature/login` , `feature/dashboard` , `feature/api`) are created and pushed to the remote repository.

```
$ git branch -a # Lists all local and remote branches
$ git push -u origin feature/login
$ git push -u origin feature/dashboard
$ git push -u origin feature/api
```

```
mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git branch -a
  feature/api
  feature/dashboard
  feature/login
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git push -u origin feature/login
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/login' on GitHub by vi
siting:
remote:      https://github.com/shmWorks/MLOPs_lab8/pull/new/feat
ure/login
remote:
To https://github.com/shmWorks/MLOPs_lab8.git
 * [new branch]      feature/login -> feature/login
branch 'feature/login' set up to track 'origin/feature/login'.

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git branch -a
  feature/api
  feature/dashboard
  feature/login
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/feature/login
  remotes/origin/main

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git push -u origin feature/dashboard
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/dashboard' on GitHub by visiting:
remote:      https://github.com/shmWorks/MLOPs_lab8/pull/new/feature/dashboard
remote:
To https://github.com/shmWorks/MLOPs_lab8.git
 * [new branch]      feature/dashboard -> feature/dashboard
branch 'feature/dashboard' set up to track 'origin/feature/dashboard'.

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git push -u origin feature/api
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/api' on GitHub by visiting:
remote:      https://github.com/shmWorks/MLOPs_lab8/pull/new/feature/api
remote:
To https://github.com/shmWorks/MLOPs_lab8.git
 * [new branch]      feature/api -> feature/api
branch 'feature/api' set up to track 'origin/feature/api'.
```

The branches are now visible on the GitHub repository.



5. Modify the Same File on a Feature Branch

On the `feature/login` branch, the `housepk_app.py` file is modified. The `git diff` command shows the changes made.

```
$ git diff --staged
```

These changes are then staged, committed, and pushed to the remote `feature/login` branch.

```
$ git add housepk_app.py
$ git commit -m "Added login-specific print and comment to index route"
$ git push
```

```
mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/login)
$ git diff --staged
diff --git a/housepk_app.py b/housepk_app.py
index 7de8761..88aec5c 100644
--- a/housepk_app.py
+++ b/housepk_app.py
@@ -1,4 +1,3 @@
-# app.py
 import os
 import joblib
 import numpy as np
@@ -40,6 +39,8 @@ app.config['TEMPLATES_AUTO_RELOAD'] = True

 @app.route("/", methods=["GET"])
 def index():
+    print("Login feature: Rendering home page")  # Added for login feature
+    # Login-specific template logic
     return render_template("index.html", feature_meta=feature_meta)

 @app.route("/predict", methods=["POST"])

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/login)
$ git add housepk_app.py

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/login)
$ git status
On branch feature/login
Your branch is up to date with 'origin/feature/login'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   housepk_app.py


mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/login)
$ git commit -m "Added login-specific print and comment to index route"
[feature/login b7a178e] Added login-specific print and comment to index route
 1 file changed, 2 insertions(+), 1 deletion(-)

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/login)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 440 bytes | 146.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/shmWorks/MLOPs_lab8.git
   f008683..b7a178e  feature/login -> feature/login
```

6. Merging a Feature Branch into `main`

The `main` branch is updated by merging the changes from `feature/login`. This is a "fast-forward" merge because there are no conflicting changes on `main`.

```
$ git switch main
$ git merge feature/login
```

```
mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git switch main
Already on 'main'
Your branch is up to date with 'origin/main'.

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git merge feature/login
Updating f008683..b7a178e
Fast-forward
 housepk_app.py | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

7. Introducing a Conflicting Change

Next, another developer merges changes from the `feature/dashboard` branch into `main`. This branch contains changes to the *same lines* of the `housepk_app.py` file that were modified in `feature/login`.

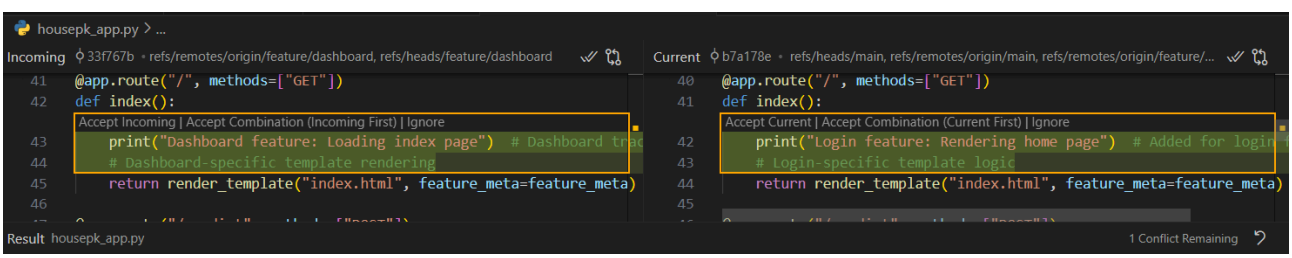
```
$ git merge feature/dashboard
```

This results in a merge conflict because Git cannot automatically decide which changes to keep.

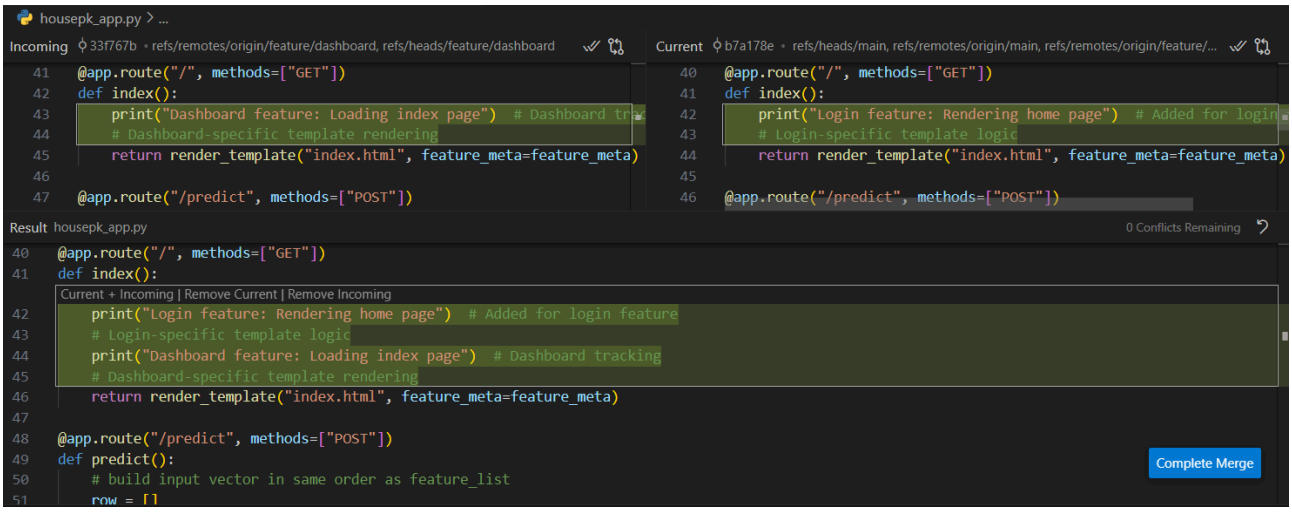
```
mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/dashboard)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ pull
bash: pull: command not found

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git pull
Already up to date.

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git merge feature/dashboard
Auto-merging housepk_app.py
CONFLICT (content): Merge conflict in housepk_app.py
Automatic merge failed; fix conflicts and then commit the result.

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main|MERGING)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   housepk_app.py

no changes added to commit (use "git add" and/or "git commit -a")
```

8. Identifying and Resolving the Merge Conflict

The file is now in a conflicted state, marked by Git. The conflict markers (`<<<<<<<` , `=======` , `>>>>>>>`) show the different versions of the code from the `main` branch (Current) and the `feature/dashboard` branch (Incoming).



To resolve the conflict, the developer manually edits the file to combine the desired changes from both branches. In this case, both print statements are kept.



9. Committing and Pushing the Resolved Code

After resolving the conflict, the file is staged and committed to finalize the merge.

```
$ git add housepk_app.py
$ git commit -m "Resolved merge conflict between feature-login and feature-das
$ git push
```

```
mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main|MERGING)
$ git add housepk_app.py

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main|MERGING)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   housepk_app.py


mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main|MERGING)
$ git commit -m "Resolved merge conflict between feature-login and feature-dashboard"
[main 40d4bd4] Resolved merge conflict between feature-login and feature-dashboard

mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 437 bytes | 145.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/shmWorks/MLOPs_lab8.git
   b7a178e..40d4bd4  main -> main
```

10. Merging Another Feature Branch

Finally, the `feature/api` branch is merged into `main`. Since the previous conflict was resolved, this merge might be a fast-forward or require another resolution depending on its changes. The image shows the process of switching branches and applying changes.

```
mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git switch feature/api
Switched to branch 'feature/api'
Your branch is up to date with 'origin/feature/api'.

mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/api)
$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 4.35 KiB | 120.00 KiB/s, done.
From https://github.com/shmworks/MLOPS_lab8
   f008683..74a761b  feature/api       -> origin/feature/api
   33f767b..9d73487  feature/dashboard -> origin/feature/dashboard
   b7a178e..6527272  feature/login     -> origin/feature/login
   40d4bd4..d751d74  main              -> origin/main
Updating f008683..74a761b
Fast-forward
 housepk_app.py | 5 ++++-
 1 file changed, 4 insertions(+), 1 deletion(-)

mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/api)
$ git add housepk_app.py

mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/api)
$ git diff

mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/api)
$ git diff --staged
diff --git a/housepk_app.py b/housepk_app.py
index fbcd842..a398ba5 100644
--- a/housepk_app.py
+++ b/housepk_app.py
@@ -39,10 +39,8 @@ app.config['TEMPLATES_AUTO_RELOAD'] = True

 @app.route("/", methods=["GET"])
 def index():
-    print("Login feature: Rendering home page")  # Added for login feature
-    # Login-specific template logic
-    print("Dashboard feature: Loading index page")  # Dashboard tracking
-    # Dashboard-specific template rendering
+    print("API feature: Serving index page")  # API-specific logging
+    # API-focused index page
     return render_template("index.html", feature_meta=feature_meta)

 @app.route("/predict", methods=["POST"])

mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/api)
$ git commit -m "Added API-specific print and comment to index route"
[feature/api 7aa9cbc] Added API-specific print and comment to index route
 1 file changed, 2 insertions(+), 4 deletions(-)

mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/api)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 400 bytes | 200.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/shmworks/MLOPS_lab8.git
   74a761b..7aa9cbc  feature/api -> feature/api

mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (feature/api)
$ git switch main
Switched to branch 'main'
Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded.
  (use "git pull" to update your local branch)

mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git pull
Updating 40d4bd4..d751d74
Fast-forward

mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git merge feature/api
Updating d751d74..7aa9cbc
Fast-forward
 housepk_app.py | 6 ++----
 1 file changed, 2 insertions(+), 4 deletions(-)

mypc@DESKTOP-OIlMDv0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git push
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/shmworks/MLOPS_lab8.git
   d751d74..7aa9cbc  main -> main
```

After all branches are merged and conflicts are resolved, the `main` branch contains the combined work of all developers. All team members can then pull the updated `main` branch to synchronize their local repositories.

```
$ git pull
```

```
mypc@DESKTOP-OI1MDV0 MINGW64 ~/Downloads/MLOPS_lab8 (main)
$ git pull
Already up to date.
```